Introducción a la Ciencia de Datos

Maestría en Ciencias de la Computación

Dr. Irvin Hussein López Nava



04

Clasificación


NO DISPAREN, SOY EXPERTO EN ML
PRUÉBALO: ¿QUÉ MODELOS APLICAS
PARA CADA PROBLEMA?
NAIVE BAYES

4.3 Bayesian learning


¿Qué es el Aprendizaje Bayesiano?

Definiciones

ChatGPT

Es un enfoque estadístico y de aprendizaje automático que se basa en el teorema de Bayes para actualizar y refinar las creencias o estimaciones a medida que se obtiene nueva evidencia. Se llama "bayesiano" en honor al estadístico británico Thomas Bayes.

https://chat.openai.com/

Wikipedia

Es un método de inferencia estadística en el que se utiliza el teorema de Bayes para actualizar la probabilidad de una hipótesis a medida que hay más evidencia o información disponible.

https://en.wikipedia.org/wiki/Bayesian_inference

Gemini

Es un enfoque de aprendizaje automático que utiliza la teoría de la probabilidad bayesiana para aprender de los datos. La teoría bayesiana ofrece un marco para calcular la probabilidad de una hipótesis dada la evidencia.

https://gemini.google.com/

Dos razones para usar aprendizaje Bayesiano

Proporciona algoritmos de aprendizaje prácticos:

- Aprendizaje Naive Bayes.
- Aprendizaje de Redes Bayesianas.
- Combina conocimientos previos (probabilidades previas) con datos observados.
- Requiere probabilidades previas.

Proporciona un marco conceptual útil:

- Proporciona un "*gold standard*" para evaluar otros algoritmos de aprendizaje.
- Información adicional sobre Occam’s razor.

Teorema de Bayes

$$P(h \mid D) = \frac{P(D \mid h)\, P(h)}{P(D)}$$

- P(h): probabilidad **a priori** de la hipótesis *h*, antes de ver los datos.
- P(D | h): **verosimilitud**, la probabilidad de observar *D* si *h* fuera cierta.
- P(h | D): probabilidad **a posteriori** de *h*, después de ver *D*.
- P(D): probabilidad de observar *D* con cualquier hipótesis. No depende de *h*.

Eligiendo las hipótesis

- Generalmente se prefiere la hipótesis más probable dados los datos de entrenamiento: la hipótesis **máxima a posteriori**, h_{MAP}:

$$
\begin{aligned}
h_{MAP} &\equiv \arg\max_{h \in H} P(h \mid D) \\
&= \arg\max_{h \in H} \frac{P(D \mid h)\, P(h)}{P(D)} \\
&= \arg\max_{h \in H} P(D \mid h)\, P(h)
\end{aligned}
$$

  - P(D) se elimina porque no depende de *h*.

- Si todas las hipótesis son igual de probables a priori, $P(h_i) = P(h_j)$, basta la verosimilitud: la hipótesis de **máxima verosimilitud**, h_{ML}:

$$
h_{ML} \equiv \arg\max_{h \in H} P(D \mid h)
$$

Ejemplo

- ¿El paciente tiene cáncer o no?
- Un paciente se hace una prueba de laboratorio y el resultado es positivo. La prueba arroja un resultado positivo correcto sólo en el 98% de los casos en los que la enfermedad está realmente presente, y un resultado negativo correcto en sólo el 97% de los casos en los que la enfermedad no está presente. Además, el 0.008 de toda la población padece este cáncer.

P(cáncer) =
P(+ | cáncer) =
P(+ | ¬cáncer) =

P(¬cáncer) =
P(- | cáncer) =
P(- | ¬cáncer) =

Ejemplo

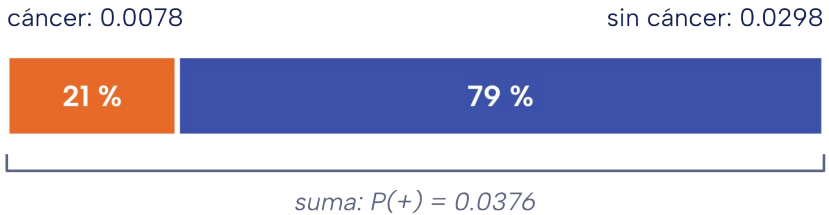
- ¿El paciente tiene cáncer o no?
- Un paciente se hace una prueba de laboratorio y el resultado es positivo. La prueba arroja un resultado positivo correcto sólo en el 98% de los casos en los que la enfermedad está realmente presente, y un resultado negativo correcto en sólo el 97% de los casos en los que la enfermedad no está presente. Además, el 0.008 de toda la población padece este cáncer.

P(cáncer) = 0.008
P(+ | cáncer) = 0.98
P(+ | ¬cáncer) = 0.03

P(¬cáncer) = 0.992
P(- | cáncer) = 0.02
P(- | ¬cáncer) = 0.97

¿El paciente tiene cáncer?

- P(+ | cáncer) P(cáncer) = 0.98 × 0.008 = **0.0078**
- P(+ | ¬cáncer) P(¬cáncer) = 0.03 × 0.992 = **0.0298**
- h_{MAP} = ¬cáncer: lo más probable es que **no** tenga cáncer.
- Para la probabilidad, se divide entre la suma:
  P(cáncer | +) = 0.0078 / (0.0078 + 0.0298) = **0.21**.



La prueba acierta en 98 % de los enfermos, y aun así un positivo deja solo 21 % de probabilidad de cáncer.

Fórmulas básicas

- Regla del producto: probabilidad de una conjunción de dos eventos A y B:

$$P(A \wedge B) = P(A|B)\, P(B) = P(B|A)\, P(A)$$

- Regla de la suma: probabilidad de una disyunción de dos eventos A y B:

$$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$$

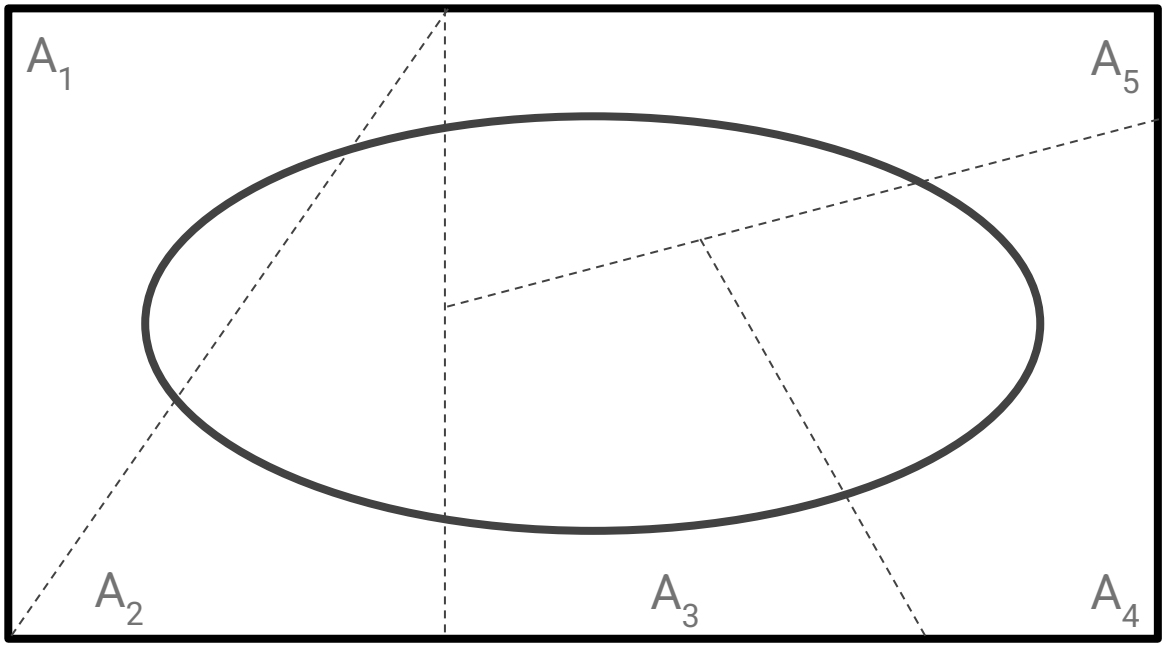
- Teorema de probabilidad total: si los eventos A_1, ..., A_n son mutuamente excluyentes con la suma $P(A_i) = 1$, entonces

$$P(B) = \sum_{i=1}^{n} P(B|A_i)\ P(A_i)$$

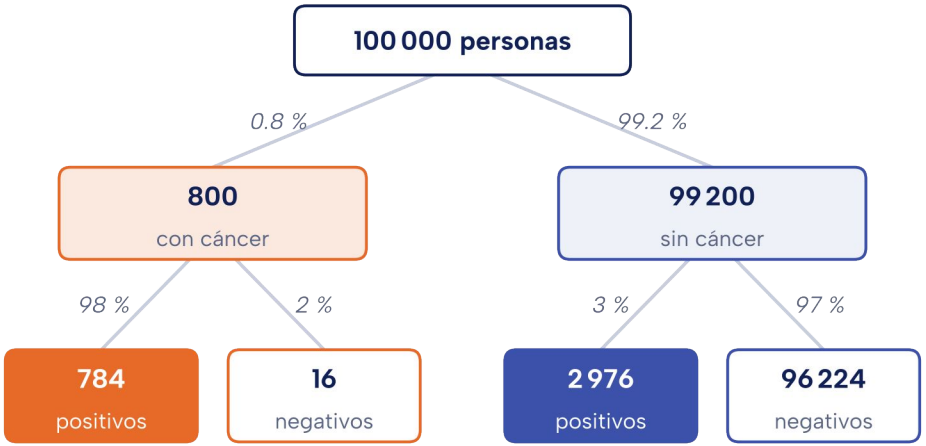
Teorema de la probabilidad total

A

B



La probabilidad total, con personas



Positivos: 784 + 2 976 = 3 760

Con cáncer: 784 / 3 760 = 21 %

Los dos caminos a un positivo suman P(+): es la probabilidad total.

Es la matriz de confusión:

- 0.98 es la sensibilidad, 0.97 la especificidad y 0.008 la prevalencia.
- P(cáncer | +) = 0.21 es la **precisión**: 79 de cada 100 positivos son falsos.

La prueba acierta en 97 % de las personas; decir siempre “no” acierta en 99.2 %.

Hipótesis MAP a fuerza bruta

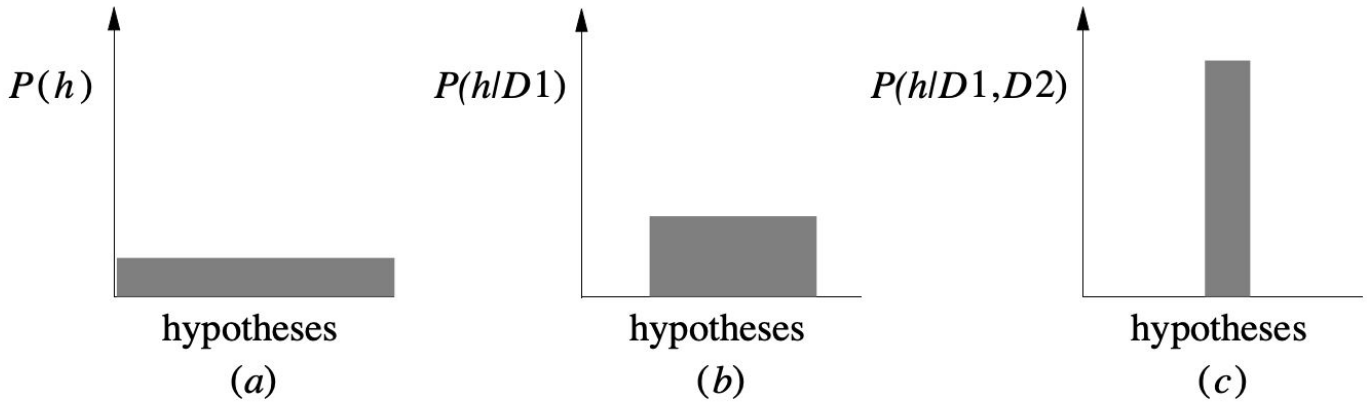
1. Para cada hipótesis h en H, calcular la **probabilidad posterior**.

$$P(h|D) = \frac{P(D|h)P(h)}{P(D)}$$

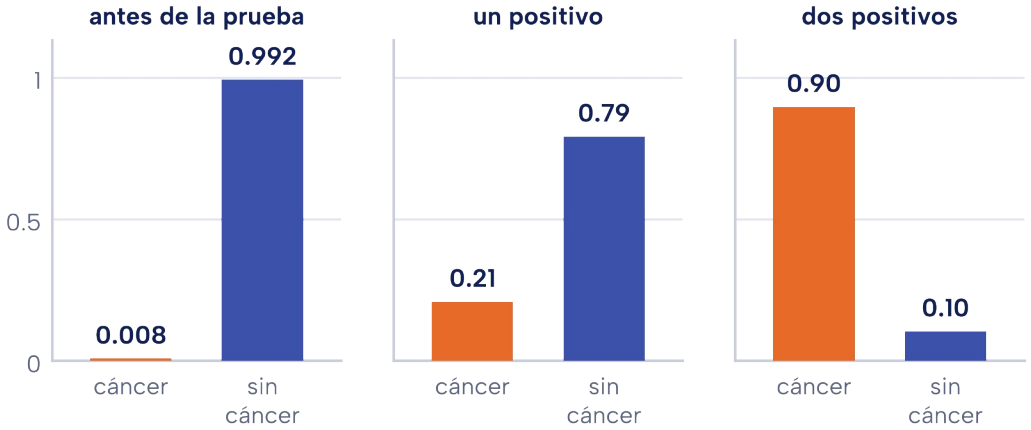
2. Devolver la hipótesis h_{MAP} con la **probabilidad posterior** más alta.

$$h_{MAP} = arg \max_{h \in H} P(h|D)$$

Evolución de las probabilidades posteriores



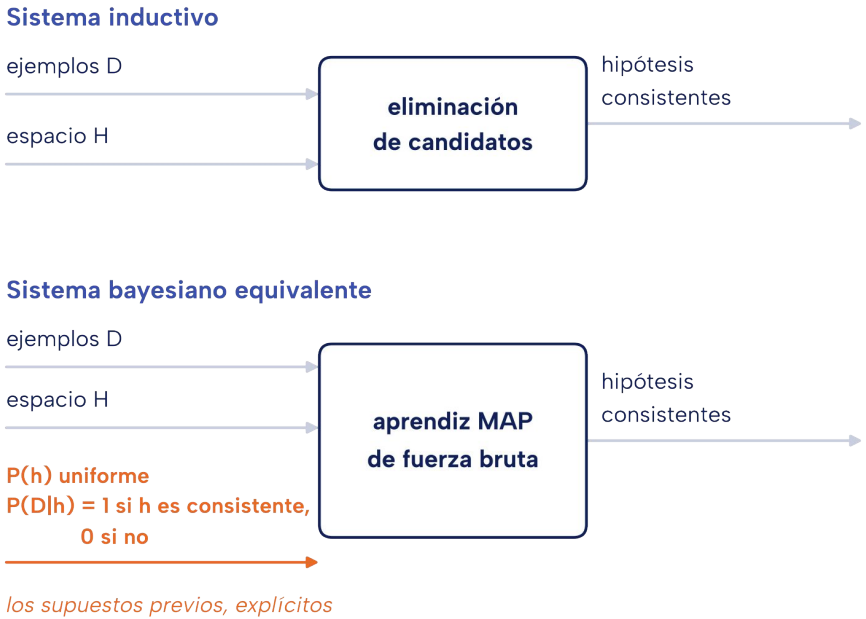
El mismo paciente, una segunda prueba



- Después del primer positivo, 0.21 es la nueva probabilidad a priori.
- Con un segundo positivo sube a **0.90**; con un tercero, a 0.996.
- Cada dato concentra la probabilidad en menos hipótesis.

Supone que las dos pruebas son independientes dado el diagnóstico: es la independencia condicional que usa Naive Bayes.

Algoritmos de aprendizaje y aprendices MAP



- Arriba, un algoritmo que conserva todas las hipótesis consistentes con los datos.
- Abajo, el aprendiz MAP de fuerza bruta, con dos supuestos:
  - P(h) uniforme;
  - P(D | h) = 1 si *h* es consistente con *D*, y 0 si no.
- Entonces toda hipótesis consistente tiene la misma posterior y las demás, 0: los dos sistemas dan las mismas hipótesis.

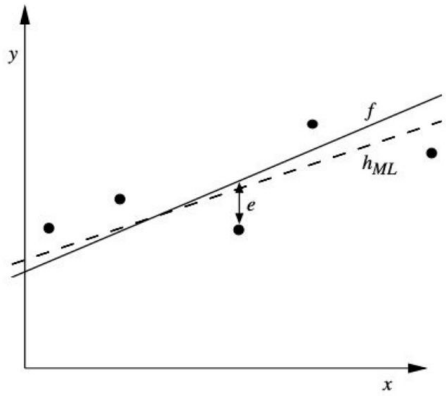
Aprender una función con valor real

Función objetivo *f* de valor real; ejemplos $\{x_i, d_i\}$, con $d_i = f(x_i) + e_i$:

- e_i es ruido independiente, normal, con media 0 y la misma varianza para todos.

La hipótesis de **máxima verosimilitud** es la que minimiza la suma de errores al cuadrado:

$$h_{ML} = \arg\min_{h \in H} \sum_{i=1}^{m} (d_i - h(x_i))^2$$



Fuente: Mitchell (1997), cap. 6.

- Es la regresión por **mínimos cuadrados** de la sesión 10: minimizar el error cuadrático es suponer ruido normal.
- Con un desenlace binario, la máxima verosimilitud lleva a la entropía cruzada: la regresión logística de la sesión 11.

Clasificación más probable de nuevas instancias

Hasta ahora se ha buscado la hipótesis más probable dados los datos D (h_{MAP}).

- Dada la nueva instancia **x**, ¿cuál es su clasificación más probable?
- ¡$h_{MAP}(x)$ no es la clasificación más probable!

Considerar:

- Tres posibles hipótesis: $P(h_1 \mid D) = 0.4$, $P(h_2 \mid D) = 0.3$, $P(h_3 \mid D) = 0.3$
- Dada la nueva instancia **x**, $h_1(x) = +$, $h_2(x) = -$, $h_3(x) = -$
- ¿Cuál es la clasificación más probable de **x**?

Clasificador óptimo de Bayes

- Ejemplo:

$$P(h_1|D) = 0.4,\ \ P(-|h_1) = 0,\ \ P(+|h_1) = 1$$
$$P(h_2|D) = 0.3,\ \ P(-|h_2) = 1,\ \ P(+|h_2) = 0$$
$$P(h_3|D) = 0.3,\ \ P(-|h_3) = 1,\ \ P(+|h_3) = 0$$

- por lo tanto

$$\sum_{h_i \in H} P(+|h_i)P(h_i|D) = 0.4$$
$$\sum_{h_i \in H} P(-|h_i)P(h_i|D) = 0.6$$

- y

$$arg\ \max_{v_j \in V} \sum_{h_i \in H} P(v_j|h_i)P(h_i|D) = -$$

Clasificador Naive Bayes

- Junto con los árboles de decisión, las redes neuronales y los k-vecinos más cercanos, es uno de los métodos de aprendizaje más prácticos.
- Cuándo usar:
  - Conjunto de entrenamiento moderado o grande.
  - Los atributos son condicionalmente independientes dada la clase; aun cuando no lo son, suele clasificar bien.
- Algunas aplicaciones exitosas:
  - Diagnóstico médico.
  - Clasificación de documentos de texto.

Formulación

- Suponga funciones objetivo $f : X \rightarrow V$, donde cada instancia x se describe con los atributos a_1, a_2, …, a_n. El valor más probable de $f(x)$ es:

$$\begin{aligned} v_{MAP} &= \arg\max_{v_j \in V} P(v_j \mid a_1, a_2, \ldots, a_n) \\ &= \arg\max_{v_j \in V} \frac{P(a_1, a_2, \ldots, a_n \mid v_j)\, P(v_j)}{P(a_1, a_2, \ldots, a_n)} \\ &= \arg\max_{v_j \in V} P(a_1, a_2, \ldots, a_n \mid v_j)\, P(v_j) \end{aligned}$$

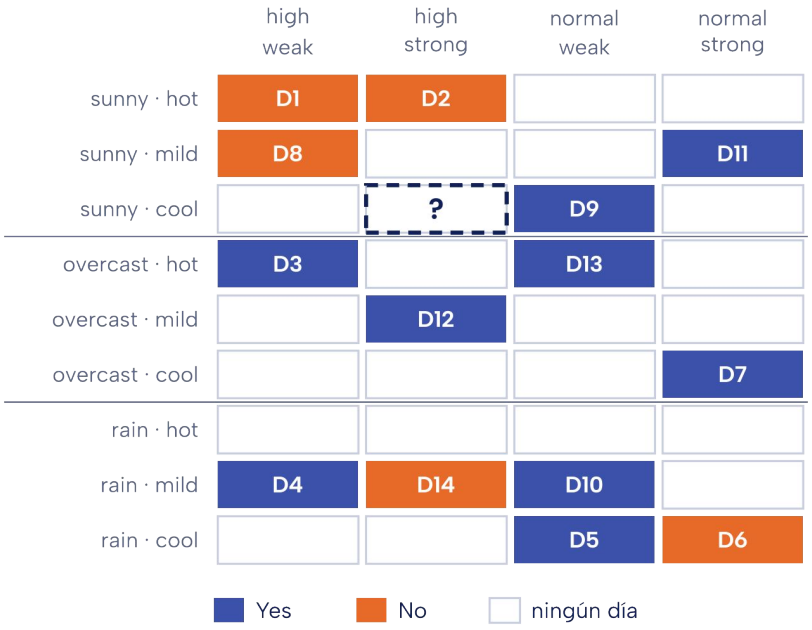
- **Suposición ingenua**: los atributos son condicionalmente independientes dada la clase: $P(a_1, a_2, \ldots, a_n \mid v_j) = \prod_i P(a_i \mid v_j)$

- lo que da el clasificador Naive Bayes:

$$v_{NB} = \arg\max_{v_j \in V} P(v_j) \prod_i P(a_i \mid v_j)$$

¿Por qué hace falta la suposición?

- Sin ella, $P(a_1, \ldots, a_n \mid v_j)$ necesita una probabilidad para cada combinación de valores.
- En *play tennis* hay $3 \times 3 \times 2 \times 2 = \mathbf{36}$ combinaciones y **14** días: cada combinación aparece una vez o ninguna.
- La instancia (sunny, cool, high, strong) no aparece: su estimación vale 0 para las dos clases.
- Con la suposición bastan 20 probabilidades $P(a_i \mid v_j)$, y cada una se estima con los 9 o 5 días de su clase.



Algoritmo Naive Bayes

Naive_Bayes_Learn(examples)

For each target value v_j

$$\hat{P}(v_j) \leftarrow estimate\ P(v_j)$$

For each attribute value a_i of each attribute a

$$\hat{P}(a_i|v_j) \leftarrow estimate\ P(a_i|v_j)$$

Classify_New_Instance(x)

$$v_{NB} = arg \max_{v_j \in V} \hat{P}(v_j) \prod_{a_i \in x} \hat{P}(a_i|v_j)$$

Nuestro ejemplo

day	outlook	temp	humidity	wind	play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Aprender es contar

Naive_Bayes_Learn estima con frecuencias: $P(v_j)$ es la fracción de días de cada clase, y $P(a_i \mid v_j)$, la fracción de los días de esa clase que tienen el valor a_i.

	Yes (9)	No (5)
Outlook = sunny	**2/9**	**3/5**
Outlook = overcast	4/9	**0/5**
Outlook = rain	3/9	2/5
Temperature = hot	2/9	2/5
Temperature = mild	4/9	2/5
Temperature = cool	**3/9**	**1/5**

	Yes (9)	No (5)
Humidity = high	**3/9**	**4/5**
Humidity = normal	6/9	1/5
Wind = weak	6/9	2/5
Wind = strong	**3/9**	**3/5**
P(v)	**9/14**	**5/14**

Nuestro ejemplo con Naive Bayes

- Considere una nueva instancia: {outlook = sunny, temperature = cool, humidity = high, wind = strong}
- Se quiere calcular: $$v_{NB} = \arg\max_{v_j \in V} P(v_j) \prod_i P(a_i \mid v_j)$$

- *P(yes) P(sunny | yes) P(cool | yes) P(high | yes) P(strong | yes) =*
- *P(no) P(sunny | no) P(cool | no) P(high | no) P(strong | no) =*
- v_{NB} *= ?*

Nuestro ejemplo con Naive Bayes

- Considere una nueva instancia: {outlook = sunny, temperature = cool, humidity = high, wind = strong}
- Se quiere calcular: $$v_{NB} = \arg\max_{v_j \in V} P(v_j) \prod_i P(a_i \mid v_j)$$

- P(yes) P(sunny | yes) P(cool | yes) P(high | yes) P(strong | yes)
  = 9/14 · 2/9 · 3/9 · 3/9 · 3/9 = **0.0053**
- P(no) P(sunny | no) P(cool | no) P(high | no) P(strong | no)
  = 5/14 · 3/5 · 1/5 · 4/5 · 3/5 = **0.0206**
- v_{NB} = **no**. Normalizando: P(no | x) = 0.0206 / (0.0053 + 0.0206) = 0.795.

El árbol de la sesión 13 también dice No: Outlook = Sunny y Humidity = High.

Sutilezas de Naive Bayes

1. A menudo se viola el supuesto de independencia condicional

$$P(a_1, a_2, ..., a_n|v_j) = \prod_i P(a_i|v_j)$$

... pero de todos modos funciona sorprendentemente bien. Tener en cuenta que no es necesario que los posteriores estimados sean correctos; solo se necesita

$$arg\max_{v_j \in V} \hat{P}(v_j) \prod_i \hat{P}(a_i|v_j) = arg\max_{v_j \in V} P(v_j) \;\; P(a_1, ..., a_n|v_j)$$

Los posteriores Naive Bayes a menudo se acercan de manera poco realista a 1 o 0.

Sutilezas de Naive Bayes

2. ¿Qué pasa si ninguna de las instancias de entrenamiento con el valor objetivo v_j tiene el valor del atributo a_i? Entonces

$$\hat{P}(a_i \mid v_j) = 0 \quad \Rightarrow \quad \hat{P}(v_j) \prod_i \hat{P}(a_i \mid v_j) = 0$$

La solución típica es la estimación bayesiana con el **estimador m**:

- *n*: número de ejemplos de entrenamiento con v = v_j;
- n_c: número de ejemplos con v = v_j y a = a_i;
- *p*: estimación previa de P(a_i | v_j);
- *m*: peso de la estimación previa (ejemplos «virtuales»).

$$\hat{P}(a_i \mid v_j) \leftarrow \frac{n_c + m\,p}{n + m}$$

Con $p = 1/k$ y $m = k$, donde *k* es el número de valores del atributo, queda la **corrección de Laplace**:

$$\hat{P}(a_i \mid v_j) \leftarrow \frac{n_c + 1}{n + k}$$

El cero está en nuestros datos

- Los cuatro días nublados son Yes: P(overcast | no) = **0/5**.
- Sin suavizar, todo día nublado sale Yes con probabilidad 1, digan lo que digan los otros atributos.

Día	P(No | x), sin suavizar	P(No | x), con Laplace
overcast, hot, high, strong	**0**	**0.436**
sunny, cool, high, strong	0.795	0.720

- En el día nublado, caluroso, húmedo y con viento, los otros tres atributos solos dan 0.81 de No. El cero de overcast lo borra; con Laplace queda en 0.436

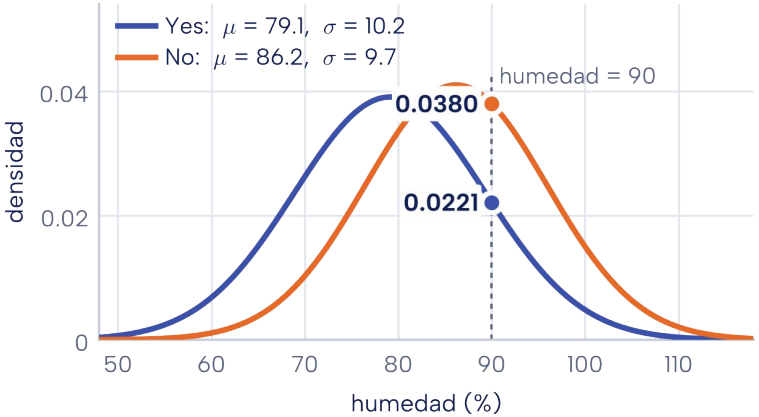
Laplace es lo que hace scikit-learn por omisión.

Atributos continuos

- Con atributos numéricos se supone una distribución normal en cada clase:

$$f(x \mid v_j) = \frac{1}{\sqrt{2\pi}\ \sigma_j} \exp\left(-\frac{(x-\mu_j)^2}{2\,\sigma_j^2}\right)$$

  - μ_j y σ_j: media y desviación estándar del atributo en los días de la clase v_j.
- Un día *sunny*, 66 ºF, humedad de 90 % y viento *strong*:
  - Yes: 9/14 · 2/9 · 3/9 · 0.0340 · 0.0221 = 0.000036
  - No: 5/14 · 3/5 · 3/5 · 0.0279 · 0.0380 = 0.000136
- P(No | x) = **0.79**.



Temperatura: medias de 73.0 y 74.6 ºF; en 66 ºF, f vale 0.0340 y 0.0279.

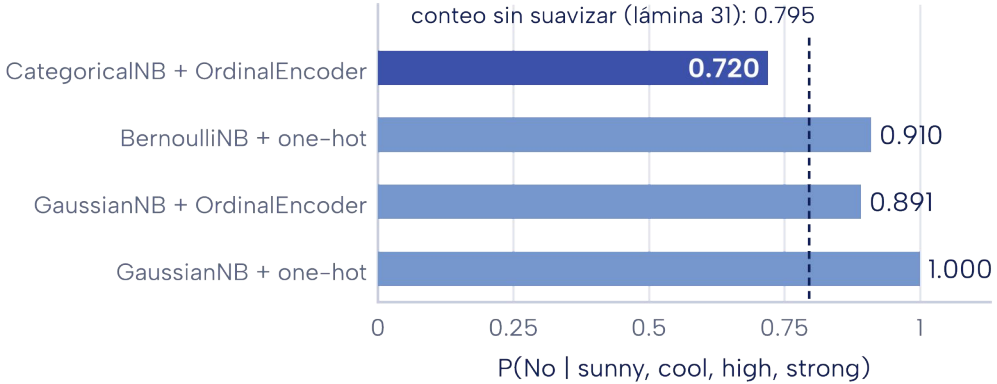
GaussianNB estima la varianza dividiendo entre n, no entre n − 1: con los mismos días da 0.81.

Naive Bayes en scikit-learn

Clase	Atributos	Cómo estima P(a | v)	Codificación
`CategoricalNB`	nominales	conteos, con suavizado α	OrdinalEncoder: los códigos son etiquetas
`GaussianNB`	continuos	una normal por clase	ninguna; no hace falta escalar
`MultinomialNB`	conteos, como las palabras de un texto	frecuencia relativa, con suavizado α	bolsa de palabras
`BernoulliNB`	binarios: presente o ausente	proporción, con suavizado α	0 y 1

- α = 1 es la corrección de Laplace; con α cerca de 0 se recupera el conteo.
- No hay una clase para atributos de los dos tipos: se combinan dos modelos o se discretizan los numéricos.
- Internamente suma logaritmos: el producto de cientos de probabilidades pequeñas, como las de las palabras de un texto, se iría a cero.

Una instancia, cuatro respuestas



- Solo la primera combinación es correcta: es la corrección de Laplace.
- Aquí `OrdinalEncoder` sí se vale: `CategoricalNB` trata los códigos como etiquetas.

- `BernoulliNB` con one-hot multiplica como independientes columnas que no lo son: si es sunny, no es rain.
- `GaussianNB` trata los códigos como números: según el orden de los códigos, da de 0.68 a 0.91.

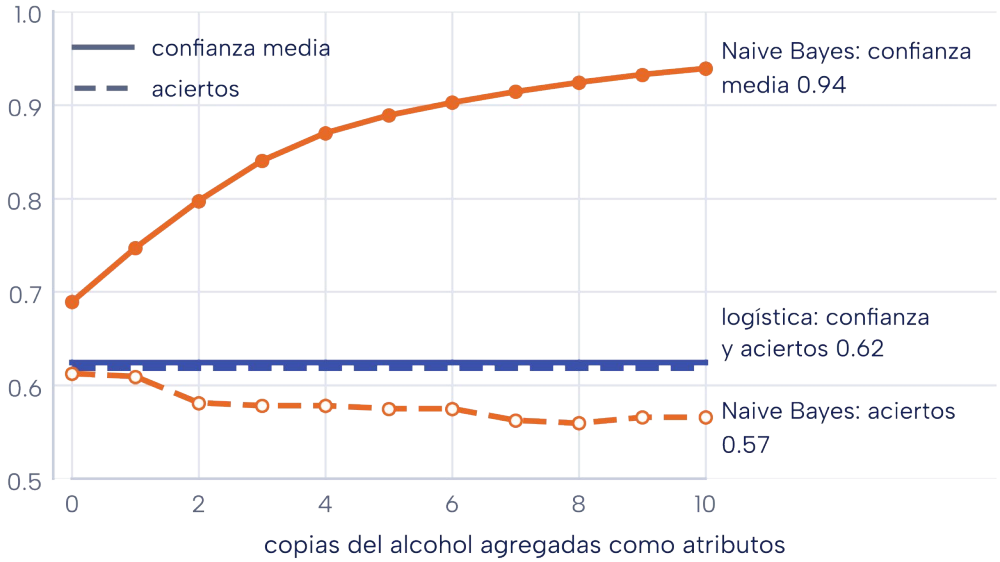
El vino: clasifica bien, estima mal

	F1 macro, validación	F1 macro, prueba	aciertos	confianza media	log-loss
Naive Bayes (GaussianNB)	**0.579**	**0.596**	**0.61**	**0.69**	**0.955**
regresión logística	0.562	0.573	0.62	0.62	0.754
árbol elegido (sesión 13)	0.584	0.551	0.58	0.76	3.133

- La suposición no se cumple: dentro de cada clase, la acidez fija tiene correlaciones de 0.63 a 0.78 (en valor absoluto) con la densidad y el pH.
- Aun así clasifica como los otros dos: las diferencias de F1 son del tamaño de la desviación entre pliegues, 0.02 a 0.04.
- Pero confía en 0.69 cuando acierta 0.61. La logística, ajustada por máxima verosimilitud (diapositiva 21), es la mejor calibrada.

Aciertos y confianza media en los 320 vinos de prueba. log-loss: entropía cruzada en prueba; menor es mejor.

La misma evidencia, contada varias veces



- Agregamos copias del alcohol como atributos: no aportan nada nuevo.
- Naive Bayes las cree independientes y cuenta la misma evidencia una y otra vez: su confianza sube a 0.94 mientras sus aciertos bajan a 0.57.
- A la logística no le pasa nada: reparte el peso entre las copias.

Redes Bayesianas

Interesantes porque:

- La suposición de Naive Bayes sobre la independencia condicional es demasiado restrictiva.
- Pero es intratable sin algunas de esas suposiciones...
- Las redes de creencia Bayesianas (también conocidas como redes Bayesianas) describen la **independencia condicional** entre subconjuntos de variables.
- Permite combinar conocimientos previos sobre (in)dependencias entre variables con datos de entrenamiento observados.

Independencia condicional

- **Definición**: X es condicionalmente independiente de Y dado Z si la distribución de probabilidad que rige a X es independiente del valor de Y dado el valor de Z; i.e., si

$$(\forall x_i, y_j, z_k) P(X = x_i | Y = y_j, Z = z_k) = P(X = x_i | Z = z_k)$$

- De manera más compacta

$$P(X|Y, Z) = P(X|Z)$$

Independencia condicional

Ejemplo:

- *Los truenos* son condicionalmente independientes de *la lluvia*, dados los relámpagos:

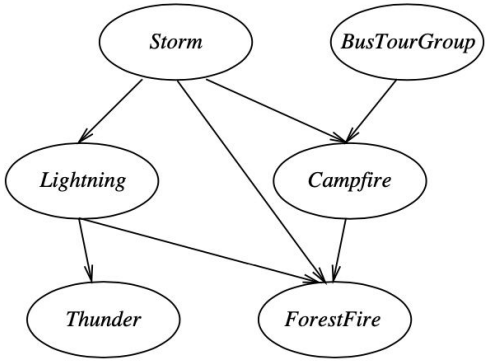
P(Thunder | Rain, Lightning) = P(Thunder | Lightning)

Naive Bayes usa la independencia condicional para justificar:

$$P(X, Y \mid Z) = P(X \mid Y, Z)\ P(Y \mid Z) = P(X \mid Z)\ P(Y \mid Z)$$

Es lo que supusimos con las dos pruebas del paciente: dado el diagnóstico, el resultado de una no cambia la probabilidad de la otra.

Red de creencia Bayesiana



	S,B	*S,¬B*	*¬S,B*	*¬S,¬B*
C	0.4	0.1	0.8	0.2
¬C	0.6	0.9	0.2	0.8

Campfire

La red representa un conjunto de afirmaciones de **independencia condicional**:

- Se afirma que cada nodo es condicionalmente independiente de sus no descendientes, dados sus predecesores inmediatos.
- Gráfico Acíclico Dirigido (DAG).

Red de creencia bayesiana

Representa la distribución conjunta de todas las variables, por ejemplo P(Storm, BusTourGroup, …, ForestFire):

$$P(y_1, \ldots, y_n) = \prod_{i=1}^{n} P(y_i \mid \text{Parents}(Y_i))$$

- Parents(Y_i) son los predecesores inmediatos de Y_i. El grafo y las tablas $P(y_i \mid \text{Parents}(Y_i))$ definen la conjunta.
- Con 6 variables binarias, la conjunta completa necesita $2^6 - 1 =$ **63** probabilidades; la red, **18**.

Naive Bayes, con ForestFire como clase, necesita 11.

Variable	Padres	Probabilidades
Storm	—	1
BusTourGroup	—	1
Lightning	*Storm*	2
Campfire	*Storm, BusTourGroup*	4
Thunder	*Lightning*	2
ForestFire	*Storm, Lightning, Campfire*	8
Total		**18**

Inferencia en las BNs

¿Cómo se pueden **inferir** las (probabilidades de) valores de una o más variables de la red, dados los valores observados de otras?

- Bayes Net contiene toda la información necesaria para esta inferencia.
- Si solo hay una variable con valor desconocido, es fácil inferirla.
- En el caso general, el problema es NP-hard.

En la práctica, puede tener éxito en muchos casos:

- Los métodos de inferencia exacta funcionan bien para algunas estructuras de red.
- Los métodos de Monte Carlo "simulan" la red aleatoriamente para calcular soluciones aproximadas.

Aprendizaje de las BNs

Variantes para la tarea de aprendizaje:

- La estructura de la red puede ser conocida o desconocida.
- Los ejemplos de entrenamiento pueden proporcionar valores de todas las variables de la red, o solo de algunas.

Si se conoce la estructura y se observan todas las variables:

- Entonces es tan fácil como entrenar un clasificador Naive Bayes.

Aprendizaje de las BNs

Suponga que se conoce la estructura y que las variables son parcialmente observables.

p.ej. *observar ForestFire, Storm, BusTourGroup, Thunder, pero no Lightning, Campfire,...*

- Similar al entrenamiento de redes neuronales con unidades ocultas.
- De hecho, se pueden aprender tablas de probabilidad condicional de la red mediante el ascenso de gradiente.
- Converger a la red ***h*** que maximiza (localmente) $P(D \mid h)$.

Ascenso de gradiente para BNs

Sea w_{ijk} una entrada de la tabla de probabilidad condicional de la variable Y_i:

$$w_{ijk} = P(Y_i = y_{ij} \mid \text{Parents}(Y_i) = u_{ik})$$

- u_{ik} es una combinación de valores de los padres; por ejemplo, si Y_i = Campfire, u_{ik} podría ser {Storm = T, BusTourGroup = F}.

Realizar un ascenso de gradiente repetidamente:

1. Actualizar todas las w_{ijk} con los datos de entrenamiento *D*: $$w_{ijk} \leftarrow w_{ijk} + \eta \sum_{d \in D} \frac{P_h(y_{ij}, u_{ik} \mid d)}{w_{ijk}}$$
2. Renormalizar las w_{ijk} para asegurar que $$\sum_j w_{ijk} = 1, \qquad 0 \le w_{ijk} \le 1$$

Converge a la red h que maximiza, localmente, P(D | h).

Más sobre cómo aprender BNs

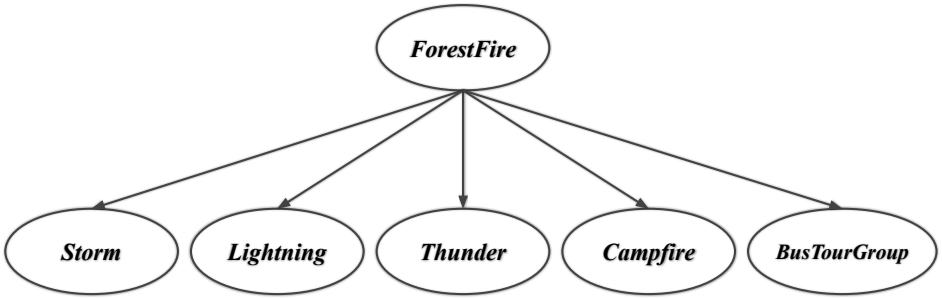
También se puede utilizar el algoritmo EM. Repetidamente:

- Calcular las probabilidades de las variables no observadas, asumiendo *h*,
- Calcular el nuevo w_{ijk} para maximizar $\mathbb{E}\,[ln\ P(D \mid h)]$, donde D ahora incluye variables observadas y (probabilidades calculadas de) no observadas.

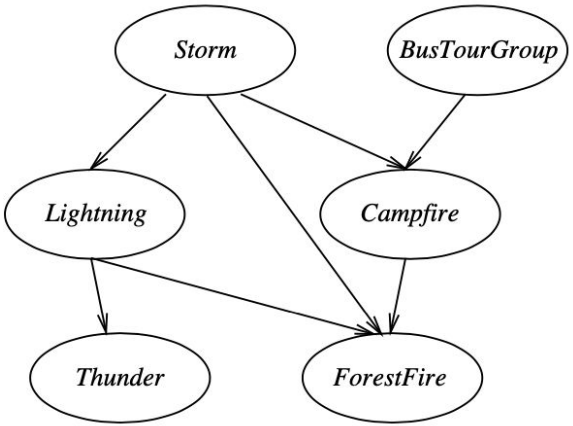
Cuando se desconoce la estructura...

- Los algoritmos utilizan búsqueda *greedy* para sumar/restar aristas y nodos.
- Es un tema de investigación activo.

Resumen: *Naive Bayes vs. Bayes Net*

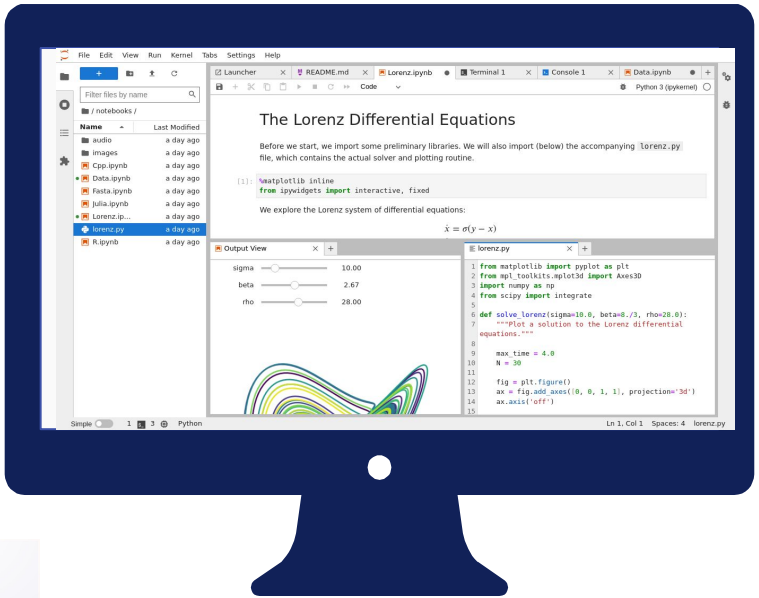


Naive Bayes

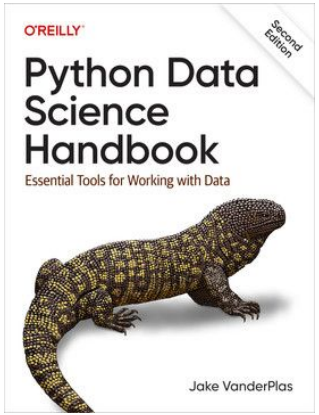


Bayes Net

(Go to live notebook)



Extra Libro



GitHub

05.05-Naive-Bayes.ipynb

Gracias!

¿Alguna pregunta?

hussein@cicese.edu.mx



SCAN ME